# NIWI-MU

## Actualización técnica

Interacciones, progreso y navegación entre escenas

Controles táctiles, audio retro y publicación Web

| | |
|---|---|
| **Motor:** | Unity 6000.3.18f1 |
| **Lenguaje:** | C# y puente JavaScript WebGL |
| **Fecha de corte:** | 18 de octubre de 2026 |
| **Base revisada:** | `main`, commit `1f1abe5` |
| **Documento:** | Versión 1.0 |

# Índice

**1 Alcance, método y estado real** **1**

**2 Organización y dependencias** **1**

2.1 Propiedad de datos y duración . . . 2

**3 Escenas y navegación por rol** **2**

3.1 Lista de compilación revisada . . . 2

3.2 Dos rutas de configuración del destino . . . 3

**4 Capas, Tilemaps y decoración** **3**

4.1 Jerarquía y orden de dibujo . . . 3

4.2 Contrato de WorldYSort . . . 3

**5 Interacción modular y creación de contenido** **4**

5.1 Procedimiento de autoría . . . 5

**6 Diálogo cinematográfico** **5**

6.1 Modelo y validación . . . 5

6.2 Cadenas disponibles . . . 6

**7 Misiones: registro implementado y motor pendiente** **6**

**8 Recompensas y promoción a Aprendiz** **6**

8.1 Definición y recurso de UI . . . 6

8.2 Máquina de estados de entrega . . . 7

8.3 Carnet de aprendiz . . . 7

**9 Teleports: identidad, acceso y transición** **7**

9.1 Contrato de SceneTeleport . . . 7

9.2 Secuencia de TeleportTravel . . . 8

9.3 Fallbacks e invariantes de llegada . . . 9

**10 Audio retro y eventos de UI** **9**

10.1 Servicios y parámetros . . . 9

10.2 Antirráfagas y prioridad . . . 10

10.3 Botones estáticos y generados . . . 10

**11 Controles móviles y responsividad** **10**

11.1 Activación y puente WebGL . . . 10

11.2 Distribución y escala . . . 11

11.3 Contrato del joystick . . . 11

11.4 Botón contextual y extensión de zonas . . . 12

**12 Frontera de replicación e integración con main** **12**

**13 Pantalla inicial y tipografía** **12**

**14 Build Web y publicación** **13**

14.1 Build manual frente a constructor de demo . . . 13

14.2 Tamaño y despliegue externo . . . 13

**15 Verificación y evidencia disponible** **14**

15.1 Comprobaciones realizadas durante las modificaciones . . . 14

15.2 Verificación reproducible del repositorio . . . 14

15.3 Matriz manual antes de una publicación . . . 15

**16 Pendientes priorizados y mantenimiento** **16**

# 1 Alcance, método y estado real

Este documento complementa la arquitectura de estados y el resumen de agosto de 2026; no reescribe su contexto histórico. El nombre visible de la aplicación es **NIWI-MU**; los namespaces, menús de creación, carpetas y GUID conservan la denominación Cenigraf para no romper referencias.

La revisión utiliza scripts C#, assets serializados, prefabs, escenas y configuración del repositorio. Se distinguen **implementado** (existe código operativo), **parcial** (existe infraestructura, pero faltan capacidades) y **pendiente** (no está implementado o falta verificarlo). Una implementación local no implica autenticación de servidor, persistencia de datos ni certificación de compatibilidad móvil.

| Elemento | Contrato y comportamiento |
|---|---|
| Interacciones y diálogos | Implementado: áreas, aviso visual, apertura con E o control táctil, escritura progresiva y recompensa al terminar. |
| Misiones | Parcial: recepción y consulta en memoria. No existen condiciones de finalización, progreso ni entrega automática por misión. |
| Recompensas y roles | Implementado: ventana, registro por ID y cambio a Aprendiz al confirmar. No equivale a un inventario persistente. |
| Teleports | Implementado: destino por escena/ID, exclusiones de rol y transición cubierta. El diseño de llegada presupone un jugador local por escena. |
| Audio | Implementado: cuatro voces sintéticas 2D, configuración de volumen y adaptación automática de botones UI. |
| Controles móviles | Implementado y probado de forma simulada. Pendiente prueba física en smartphones/tablets y medición de rendimiento. |
| Replicación | Parcial: frontera de autoridad y snapshots del jugador. La publicación Web no demuestra que el transporte de red funcione en navegador. |

# 2 Organización y dependencias

Las rutas de este manual son relativas a la raíz Unity, no al directorio padre del repositorio. La raíz de módulos es `Assets/_SocialProyectoCenigraf/Scripts`.

| Elemento | Contrato y comportamiento |
|---|---|
| `World/ModularInteraction.cs` | Detecta ocupantes, activa la misión opcional y solicita el diálogo. Implementa el contrato de interacción móvil. |
| `Dialogue/` | Catálogo de cadenas y presentación cinematográfica; no conoce el transporte de red. |
| `Missions/` | Definiciones y diario estático de misiones activas. |
| `Rewards/` | Definición de recompensa, ventana y confirmación del rol. |
| `World/Teleport/` | Área, permisos, mensaje de rechazo, carga y colocación del jugador. |
| `Audio/` | Generación de tonos, fuentes persistentes y eventos de botones. |
| `Mobile/` | HUD, joystick, registro de objetivos y arbitraje de interacción táctil. |
| `Assets/Plugins/WebGL/NiwiMobile.jslib` | Detección de dispositivo, escala CSS y desactivación del desplazamiento táctil del canvas. |
| `Player/Replication/` | Identidad/autoridad del jugador y adaptación entre snapshots y store. |

| Elemento | Contrato y comportamiento |
|---|---|
| `Editor/WebPreviewBuild.cs` | Constructor de la demo WebGL. Solo se compila dentro del editor. |

La dirección principal de dependencias es: entrada del usuario, componente de interacción, catálogo, presentador de diálogo, ventana de recompensa y stores de jugador/sesión. Las misiones son una vía independiente activada por entrada al área. Los servicios de audio no leen el teclado: los invocan acciones y botones. El joystick aporta un vector al controlador de movimiento; no modifica directamente el Transform ni simula una tecla E.

### 2.1 Propiedad de datos y duración

| Elemento | Contrato y comportamiento |
|---|---|
| `PlayerStateStore` | Posición, rol, apariencia, identidad y estado de movimiento. Los adaptadores aplican física y renderizado. |
| `GameSessionStore` | Selección de rol conservada entre escenas de la sesión. El receptor nuevo obtiene este rol al inicializarse. |
| `MissionJournal` | Diccionario estático de misiones activas; se reinicia mediante `SubsystemRegistration`. |
| `RewardWindow` | Instancia de UI ligada a escena; conjuntos estáticos `received` y `acknowledged` duran la partida. |
| Audio y controles | Objetos `DontDestroyOnLoad`; se reinician sus referencias estáticas al comenzar una sesión de ejecución. |
| Catálogos | Assets `ScriptableObject` de configuración. No son bases de datos ni archivos de guardado del progreso. |

Cerrar o recargar la aplicación reinicia misiones, recompensas y selección de sesión. La caché de descarga de un navegador no conserva esos datos de juego.

## 3 Escenas y navegación por rol

### 3.1 Lista de compilación revisada

| Elemento | Contrato y comportamiento |
|---|---|
| `Scenes/Deprecate/InitialDemoScene.unity` | Habilitada, primera en `EditorBuildSettings`. Contiene selección de roles y submenús. |
| `Scenes/Salas/Plaza_Central.unity` | Habilitada. Inicio del recorrido y diálogo del guía. |
| `Scenes/Salas/Sala_ABC.unity` | Habilitada. Sala de pruebas de interacciones, capas y teleports. |
| `Scenes/Salas/Sala_Lobby.unity` | Habilitada. Recepción y entrega del carnet. |
| `Scenes/Deprecate/SceneDemo.unity` | Deshabilitada en la lista actual; se conserva como escena anterior de integración. |

| Elemento | Contrato y comportamiento |
|---|---|
| `Scenes/SalaABC.unity` | Asset adicional recibido de otra rama. No es el mismo archivo que `Salas/Sala_ABC.unity` y no está habilitado en la lista actual. |

En esta tabla, `Scenes/` abrevia `Assets/_SocialProyectoCenigraf/Scenes/`. Los perfiles de compilación pueden introducir una lista propia; verificar el perfil activo antes de publicar.

### 3.2 Dos rutas de configuración del destino

`RoleDefinition` serializa `destinationScene` como `SceneAsset` en el editor y guarda `destinationScenePath` mediante `OnValidate`. El runtime utiliza `DestinationSceneName`. Para Visitante y Aprendiz se edita **Navigation / Destination Scene** en `Data/Roles/ROLE_Visitor.asset` o `ROLE_Apprentice.asset`. Ambos apuntan actualmente a Plaza Central.

Docente y Administrativo tienen `entryPoint=Administration`. `InitialMenuFlowController.TryHandleRole` intercepta la selección y construye un submenú en la misma escena. Por eso modificar el destino del asset de rol no cambia la opción **Ir al juego** de ese panel. Su callback está en `ShowAdministration`:

```
1 () => SceneNavigationService.Instance.NavigateTo("Plaza_Central")
```

El nombre es una constante de código en el estado actual; no existe un campo serializado para elegir este destino en el Inspector. Administrativo habilita la gestión de publicaciones; Docente no tiene ese permiso. Estas reglas son locales de interfaz.

## 4 Capas, Tilemaps y decoración

### 4.1 Jerarquía y orden de dibujo

La posición en la jerarquía organiza objetos, pero no reemplaza `SortingLayer`, `OrderinLayer` ni un `SortingGroup`. La configuración de `TagManager.asset` conserva, en orden, `Default`, `Background`, `Floor`, `WorldDynamic`, `Structure`, `Decoration`, `Player`, `Foreground` y `UI`.

Fondo y piso deben quedar detrás de los elementos dinámicos. Para que un muro, decoración o NPC alterne delante/detrás del personaje por su posición Y, sus renderers y el personaje deben participar en una capa de ordenamiento común, actualmente `WorldDynamic`. Si permanecen en capas diferentes, la prioridad de la capa prevalece sobre el número calculado por Y. Los grupos de NPC de la jerarquía no implican que exista una Sorting Layer independiente llamada NPC.

### 4.2 Contrato de WorldYSort

| Elemento | Contrato y comportamiento |
|---|---|
| `sortPoint` | Transform de referencia. Si no está asignado, busca el hijo `SortPoint`. Colocarlo en el pie/base que toca el suelo. |

| Elemento | Contrato y comportamiento |
|---|---|
| `targetSortingGroup` | Destino preferido. Si existe, aplica el orden a todo el conjunto; evita separar visualmente las partes de un objeto. |
| `targetRenderer` | Alternativa cuando no se usa un grupo. |
| `ordersPerUnit` | 100 por defecto; convierte la coordenada Y en prioridad de dibujo. |
| `orderOffset` | Desplazamiento adicional, 0 por defecto. |
| `updateContinuously` | Recalcula en `LateUpdate` cuando está habilitado. Necesario para objetos móviles si no se actualiza por otro adaptador. |
| `sortingEnabled` | Activa el cálculo dinámico. Desactivado, usa `disabledSortingOrder`. |

El orden se calcula como RoundToInt$(-y \cdot \text{ordersPerUnit}) + \text{orderOffset}$. Un valor Y menor produce un orden mayor. La posición del pivote de la imagen no sustituye un SortPoint correctamente ubicado.

Un Tilemap gobernado como una sola unidad por un Renderer o SortingGroup comparte un orden para todo ese conjunto. Para muros grandes con varios pies de profundidad, dividirlos en piezas con referencias de suelo coherentes; un único orden no expresa todas sus alturas. Revisar también el modo de renderizado del Tilemap antes de esperar ordenamiento por tile.

Las etiquetas de identificación de decoración son elementos visuales de maqueta; el texto Chair no constituye un ID de inventario ni una regla de interacción. Para reemplazar arte, preservar los componentes, las referencias y los SortPoint de cada prefab.

## 5 Interacción modular y creación de contenido

Prefab base: `Assets/_SocialProyectoCenigraf/Prefabs/Interactions/Modular_Interaction.prefab`. Requiere `BoxCollider2D`; `Reset` y `Awake` fuerzan `isTrigger=true`. El jugador necesita su controlador, collider y cuerpo físico para generar eventos de trigger.

| Elemento | Contrato y comportamiento |
|---|---|
| `interactionPrompt` | Objeto visual hijo que se muestra con el jugador en rango y se oculta durante diálogo/recompensa. No asignar el propio objeto raíz. |
| `areaColor` | Violeta `(0.65,0.55,0.78,0.18)`. Solo colorea Gizmos en Scene; no pinta el suelo del juego. |
| `dialogueCatalog`, `dialogueId` | Referencia al catálogo y cadena. Sin ambos no se abre diálogo. |
| `activateMissionOnEnter` | Desactivado por defecto. Si está activo, solicita la misión al entrar el primer collider del jugador. |
| `missionCatalog`, `missionId` | Fuente e ID de la misión opcional. Son independientes de los campos de diálogo. |

Los ocupantes se registran en un `HashSet<Collider2D>`; esto evita duplicar la entrada del mismo collider. Se limpian referencias destruidas, deshabilitadas o inactivas. Solo se aceptan controladores locales activos; una réplica remota no activa el área.

`Interact()` devuelve si se abrió un diálogo. Rechaza una UI modal abierta y el frame en que se cerró un diálogo o recompensa. La lectura de teclado también evita actuar mientras un campo de texto tiene el foco. El control móvil usa `CanInteractWith(player)` y `InteractWith(player)`; además valida que el catálogo resuelva la cadena.

### 5.1 Procedimiento de autoría

1. Instanciar el prefab y ubicarlo sobre el NPC u objeto de referencia.
2. Ajustar **Box Collider 2D / Size y Offset**. Los Gizmos ayudan a revisar la cobertura; no requieren cambiar el tamaño del Sprite.
3. Asignar el icono futuro en el objeto Image correspondiente del aviso. El arte puede sustituirse sin modificar la lógica.
4. Elegir catálogo y cadena. Para un área solo informativa, dejar desactivada la misión.
5. Si activa misión, seleccionar catálogo e ID y comprobar que no colisione con IDs de otros catálogos.
6. Guardar escena y probar entrada/salida, reapertura y cierre de UI con el jugador local.

Un área sin diálogo puede activar una misión al entrar, pero no ofrece una acción manual adicional con E en esta versión.

## 6 Diálogo cinematográfico

### 6.1 Modelo y validación

`DialogueCatalog` contiene un arreglo de `DialogueSequence`. Cada secuencia tiene `id`, `lines` y `completionReward`; cada `DialogueLine` tiene `speaker` y `text`. `TryGet` requiere un ID no vacío, una sola coincidencia y al menos una línea. La comparación del ID es exacta; no hay un resolutor global entre catálogos.

| Elemento | Contrato y comportamiento |
|---|---|
| `TryOpen(catalog,id,player)` | Rechaza diálogo/recompensa abierta, receptor nulo o catálogo inválido. Crea el presentador si hace falta y pausa el controlador del receptor. |
| `ShowLine()` | Asigna nombre y cuerpo, fija `maxVisibleCharacters=0`, actualiza la malla y registra el frame de entrada. |
| `Update()` | Aumenta caracteres con `Time.unscaledDeltaTime`; velocidad predeterminada de 35 caracteres/segundo. |
| `Advance()` | Primero completa la línea si faltan caracteres; después avanza o finaliza. No procesa la apertura y el avance en el mismo frame. |
| `AdvanceCurrent()` | Adaptador estático utilizado por el botón táctil. Reutiliza `Advance`; no duplica reglas de diálogo. |
| `Close()` | Restaura el movimiento previo, oculta la UI y fija `LastClosedFrame`. Esc cancela sin recompensa. |

La UI se construye en ejecución con `Canvas` Screen Space Overlay y orden 2000. Utiliza TMP, fuente por defecto y franjas negras. En móvil, la franja inferior ocupa el 38 % de altura en lugar del 27 % de escritorio y el texto reserva el lado derecho para E. El presentador es de escena; no tiene `DontDestroyOnLoad`.

No hay opciones de respuesta, bifurcaciones, condiciones narrativas, localización por claves ni guardado del índice de línea. No confundir `completionReward` con finalización de una misión.

### 6.2 Cadenas disponibles

El catálogo de ejemplo está en `Assets/_SocialProyectoCenigraf/Asset/Dialogue/DialogueCatalog_Example.asset`.

| Elemento | Contrato y comportamiento |
|---|---|
| `bienvenida_sala_abc` | Tres líneas de presentación del área de pruebas. |
| `bienvenida_parque_central` | Tres líneas del guía: bienvenida y orientación para recibir el carnet en Lobby. |
| `recepcion_lobby_entrega_carnet` | Cuatro líneas del recepcionista y `Reward_CarnetAprendiz` como recompensa final. |

Para crear contenido: **Create / Cenigraf / Dialogue Catalog**; agregar secuencia con ID único y líneas, asignar recompensa opcional y referenciar la cadena desde la interacción.

## 7 Misiones: registro implementado y motor pendiente

`MissionDefinition` define `id`, `title`, `description` y `objective`. Los dos últimos son texto descriptivo, no expresiones que Unity ejecute. El catálogo permite resolver una definición; no supervisa el mundo.

`MissionJournal.TryActivate(catalog,id)` valida el catálogo, rechaza una misión ya registrada y agrega la definición a un diccionario por ID. Publica `MissionActivated` y escribe un registro en consola. `IsActive(id)` y `ActiveMissions` permiten consultar el estado. La regla de unicidad es para toda la partida: dos catálogos con el mismo ID compiten por la misma entrada.

**No existe aún una condición de misión cumplida.** No se evalúan visitar una sala, hablar con un NPC, obtener el carnet, un contador, una secuencia de objetivos ni un estado Completed. Tampoco existe un HUD de misiones ni una recompensa derivada de ese estado. Para incorporarlos se necesita un modelo explícito de progreso, eventos de dominio, evaluación idempotente y persistencia. Esta es una ampliación pendiente, no una función oculta del catálogo.

## 8 Recompensas y promoción a Aprendiz

### 8.1 Definición y recurso de UI

`RewardDefinition` contiene `id`, `displayName`, `description`, `icon` y `roleOnContinue`. Se crea con **Create / Cenigraf / Reward**. `RewardWindow.GrantAndShow(reward,player)` carga `Resources/UI/RewardWindow.prefab` si no existe una instancia.

El prefab utiliza un fondo extendido horizontalmente y contenido centrado. Para cambiar su arte o distribución, editar el prefab conservando las referencias a texto, imagen y botón. Si falta el Sprite, la imagen se oculta. La ventana puede crear un EventSystem temporal si no existe ninguno.

### 8.2 Máquina de estados de entrega

1. **No recibida**: el ID no está en `received` ni en `acknowledged`.
2. **Recibida pendiente**: al abrir una entrega válida se agrega a `received`. `HasReceived(id)` ya es verdadero antes de confirmar. Solo esta primera recepción reproduce la melodía.
3. **Confirmada**: Continuar, E o Enter aplican `roleOnContinue`, si existe, y agregan el ID a `acknowledged`. Las entregas futuras de ese ID se rechazan.
4. **Cerrada sin confirmar**: Esc cierra la ventana y restaura movimiento, pero no confirma ni cambia el rol. Una repetición del diálogo puede volver a mostrar la entrega pendiente sin repetir su registro ni la melodía de adquisición.

La apertura rechaza recompensa nula, ID vacío, ID confirmado y otra recompensa ya visible. Para confirmar un cambio de rol es necesario el `PlayerStateStore` del receptor; si no existe, emite una advertencia y conserva abierta la ventana. La acción de confirmar reproduce el tono corto; cancelar con Esc no lo hace.

Los conjuntos estáticos son marcas de recepción, no cantidades, equipamiento, transacciones de servidor ni inventario general. El método público `Show` presenta la UI; la ruta de entrega con registro es `GrantAndShow`.

### 8.3 Carnet de aprendiz

El asset `Asset/Rewards/Reward_CarnetAprendiz.asset` conserva el ID estable `carnet_estudiantil`; su nombre visible es Carnet de aprendiz. `roleOnContinue` apunta a `ROLE_Apprentice`. Confirmar realiza dos actualizaciones:

```
state.SetRole("apprentice");
GameSessionStore.Instance.Dispatch(
    GameSessionAction.SetSelectedRole("apprentice"));
```

La primera afecta al jugador actual; la segunda permite que el rol se aplique al jugador de otra escena. Son dos despachos separados, no una transacción global. `RoleIds.Normalize` traduce el ID heredado `student` a `apprentice`; no se debe cambiar el GUID del asset de rol al renombrar su presentación.

El sistema anterior `Interaction_oso/ObjetoRecogible` permanece separado. Sus banderas y bloqueos no son el registro de recompensa nuevo.

## 9 Teleports: identidad, acceso y transición

### 9.1 Contrato de SceneTeleport

Prefab general: `Prefabs/Interactions/Teleport_.prefab`, relativo a `Assets/_SocialProyectoCenigraf/`. Su BoxCollider2D es trigger. El color Scene es aguamarina `(0.45,0.76,0.78,0.18)`.

| Elemento | Contrato y comportamiento |
|---|---|
| `teleportId` | String único dentro de la escena. Se recortan espacios exteriores; la comparación distingue mayúsculas y conserva ceros iniciales. `024` no es `24`. |
| `spawnPoint` | Transform opcional de llegada. Si falta, utiliza la posición del teleport. Permite separar el destino del área de salida. |
| `destinationScenePath` | Ruta de escena. El inspector personalizado la rellena al arrastrar un SceneAsset. Vacía permite un punto que solo recibe. |
| `destinationTeleportId` | ID que se busca exclusivamente en la escena de llegada. |
| `interactionPrompt` | Aviso de proximidad. Su imagen futura se asigna en `InteractionPrompt/Frame/Visual`. |
| `restrictByRole` | Activa la exclusión. Desactivado, permite todos los roles. |
| `roleCatalog` | Facilita selección visual de IDs en el Inspector; la regla runtime se evalúa contra la lista de strings. |
| `excludedRoleIds` | Lista de roles que no pueden salir por este teleport. Vacía no bloquea a nadie. |
| `accessDeniedMessage` | Texto del aviso de rechazo; no inicia transición ni cambia posición. |

`TryTravel(player)` exige jugador local activo, presencia de su collider en el área, destino no vacío y ausencia de diálogo, recompensa o viaje en curso. Consulta el rol actual del jugador en el momento de actuar; si falta usa Visitante. El mensaje de acceso denegado dura tres segundos y no pausa el movimiento.

Las exclusiones son **reglas de salida**: hay que configurar cada entrada que conduzca a una zona restringida. No son una autorización de llegada ni una medida de seguridad de servidor. La E móvil puede mostrarse ante un teleport restringido para que el usuario reciba el mensaje al intentar viajar.

## 9.2 Secuencia de TeleportTravel

1. `Begin` verifica que la escena pueda cargarse y fija `IsTraveling`. Un destino no habilitado se rechaza antes de la cobertura.
2. Crea un host persistente, pantalla negra opaca y cámara negra sin objetos visibles; espera un frame antes de iniciar `LoadSceneAsync` en modo Single.
3. En `sceneLoaded`, guarda el estado previo de cámaras, seguidores, controladores, simulación física y renderers de llegada; los suspende temporalmente.
4. Espera a que termine la carga y un frame adicional para permitir la inicialización mediante Start.
5. Busca el ID. Si existe una sola coincidencia, actualiza Rigidbody2D, Transform y store con `ReconcilePosition`; sincroniza física y refresca Y-sort.
6. `CameraFollowController.SnapToTarget` limpia historial de retardo y velocidad de suavizado, aplica límites y fija la cámara sobre el destino resuelto.
7. Mantiene como mínimo un segundo desde la cobertura inicial. Si la carga tarda más, espera; no corta la operación al cumplirse un segundo.
8. Restaura renderizado y realiza un fundido de 0.35 segundos. Al terminar restaura movimiento y física, limpia suscripciones y destruye el host.

Los tiempos se definen en `minimumLoadingSeconds` y `fadeSeconds` del componente creado por código. Cambiar un prefab de teleport no modifica esos valores de la transición.

### 9.3 Fallbacks e invariantes de llegada

| Elemento | Contrato y comportamiento |
|---|---|
| ID ausente, vacío o duplicado | Advertencia y conservación del spawn por defecto de la escena de llegada, bajo la misma cobertura negra. |
| Escena no habilitada | Se conserva la escena actual y no se inicia viaje. |
| Jugador de llegada inexistente | Advertencia; no se instancia un personaje automáticamente. |
| Varios jugadores de llegada | La colocación usa el primer jugador recogido por el gate. No hay resolución por playerId; conservar un único jugador local activo hasta ampliar este contrato. |
| Error durante carga | `finally/Finish` restaura lo que quedó registrado y limpia `IsTraveling`. |
| Navegación atrás | El servicio de navegación no permite otra carga mientras se está viajando. El teleport no incorpora su ruta a la pila de historial del menú. |

El prefab `Resources/UI/LoadingScreen.prefab` ubica el indicador abajo a la derecha: referencia 1920×1080, márgenes 48/40 y slot de icono 64×64. Asignar el Sprite en `LoadingIndicator/IconSlot/LoadingIcon`; el script habilita su Image solo si tiene Sprite.

La LoadingCamera conserva un render negro mientras las cámaras de llegada están suspendidas. Así se evita el aviso de Game View sin cámaras. Esta cobertura pertenece al viaje por teleport: no se aplica automáticamente a toda carga realizada por otros servicios.

## 10 Audio retro y eventos de UI

### 10.1 Servicios y parámetros

`RetroFeedbackAudio` se crea al cargar la primera escena y persiste entre escenas. Genera clips mono a 44100 Hz, sin archivos WAV ni descargas externas. Tiene **cuatro AudioSource**, todas 2D, sin loop ni reproducción automática. La configuración se resuelve por `Resources.Load<RetroAudioSettings>("Audio/RetroAudioSettings")`.

| Elemento | Contrato y comportamiento |
|---|---|
| `letterVolume=0.08` | Tono de 22 ms, frecuencia descendente de 620 a 540 Hz. Se solicita para letras y números revelados. |
| `interactionVolume=0.14` | Tono de 75 ms, de 740 a 980 Hz. Apertura/avance de diálogo, viaje aceptado y confirmación de recompensa; también clic/submit de botones UI. |
| `rewardVolume=0.16` | Frase original ascendente de 0.60 s; frecuencias 523.25, 659.25, 783.99, 1174.66 y 1046.50 Hz, con nota final más larga. |

| Elemento | Contrato y comportamiento |
|---|---|
| `hoverVolume=0.07` | Tono de 35 ms, de 460 a 580 Hz al entrar sobre un botón interactuable. Intervalo mínimo global de 60 ms entre hovers. |

Los volúmenes son ganancias lineales de 0 a 1, no decibelios. Se ajustan en `Resources/Audio/RetroAudioSettings.asset`. Cero silencia el efecto. La forma de onda combina la fundamental y armónicos tercero/quinto, con ataque de 3 ms, caída progresiva y ganancia interna 0.35 para evitar bordes abruptos.

### 10.2 Antirráfagas y prioridad

Como máximo se reproduce un bit de letra y una confirmación por frame. Si un frame revela varios caracteres, se solicita un solo bit; no hay garantía de un evento acústico individual por letra con FPS bajos. Espacios y signos quedan en silencio. Completar una línea con E no reproduce todos los caracteres de golpe.

`PlayReward` detiene letra y confirmación para que la adquisición se distinga. La melodía se solicita solo cuando `received.Add(id)` devuelve verdadero. El código no contiene un sample ni la melodía de Mario: es una frase propia con el mismo sintetizador del proyecto.

### 10.3 Botones estáticos y generados

`LateUpdate` consulta `Selectable.AllSelectablesNoAlloc`, reutiliza un buffer y añade `RetroButtonAudio` a los Button que no lo tengan. El componente implementa `IPointerEnterHandler`; conecta `Button.onClick` al habilitarse y lo desconecta al deshabilitarse. Por eso los menús construidos dinámicamente y el botón móvil reciben audio sin editar cada prefab.

El hover verifica `IsActive/IsInteractable`. El clic también cubre la activación de un botón mediante navegación de teclado. Las confirmaciones que coincidan en un frame comparten la deduplicación del servicio. La selección por foco sola no tiene un evento de audio específico; el hover corresponde al puntero. Se requiere un AudioListener activo y que el navegador permita audio tras un gesto del usuario.

## 11 Controles móviles y responsividad

### 11.1 Activación y puente WebGL

`MobileControls.Initialize` crea el HUD únicamente en dispositivo móvil. En aplicaciones nativas utiliza `Application.isMobilePlatform`; en WebGL llama a `NIWI_IsMobileDevice` mediante `DllImport("__Internal")`.

El plugin reconoce Android, iPhone, iPad, iPod y agentes móviles/tablet; también trata como iPad el agente Macintosh/MacIntel con más de un punto táctil. Es una heurística de navegador, no una identificación garantizada de hardware. Una computadora con pantalla táctil no activa el HUD solo por tener `maxTouchPoints`. Reducir el tamaño de ventana tampoco lo activa. No se fuerza una orientación de pantalla.

El puente aplica `touchAction="none"` al canvas en móvil para que el gesto de la palanca no desplace la página. `NIWI_CanvasPixelRatio` calcula ancho de render del canvas dividido por su ancho CSS; así los 200 píxeles lógicos no crecen de forma excesiva en una tablet Retina.

## 11.2 Distribución y escala

| Elemento | Contrato y comportamiento |
|---|---|
| Canvas del HUD | Screen Space Overlay, orden 3000; sprites circulares generados en memoria. Sin prefab obligatorio ni cambios manuales en cada escena. |
| Base del joystick | 200×200 unidades UI, abajo/izquierda, desplazamiento `(16,24)` dentro del área segura. Gris translúcido. |
| Mango | 100×100; relleno de 92×92 y borde. El rojo de la referencia no se usa como color definitivo. |
| Botón E | 100×100, abajo/derecha, desplazamiento `(-56,44)`. Tiene navegación de foco desactivada. |
| Área segura | Anchors calculados a partir de `Screen.safeArea/Screen.width/Screen.height`; se recalculan durante el juego. |
| Escala Web | Mínimo entre ratio de píxeles del canvas y lado corto seguro dividido por 402; mínimo técnico de escala 0.1. |
| Escala nativa | Densidad `Screen.dpi/160` cuando está disponible; se limita del mismo modo por área segura. Editor usa ratio 1. |

Las dimensiones son de diseño, no píxeles físicos fijos de un panel. En una superficie menor de 402 píxeles lógicos de lado corto se reduce proporcionalmente para que los controles quepan. No está implementada una personalización por usuario de tamaño, opacidad o posición.

## 11.3 Contrato del joystick

`MobileJoystick` implementa PointerDown, Drag, PointerUp e InitializePotentialDrag. Guarda el ID del primer puntero, desactiva el umbral de arrastre y rechaza intentos de otro dedo sobre la misma palanca. El botón sigue recibiendo el segundo toque por el sistema UI multitáctil.

La posición táctil se transforma al rectángulo local, se resta su centro y se limita a un radio $r = (200 - 100)/2 = 50$. El mango no sale de la base. Si la distancia es menor que $0.12r$, el vector es cero; en el resto es offset/r. Hay una zona muerta del 12 %, sin remapeo continuo de ese margen.

`PlayerMovementController.Update` da prioridad al vector táctil si es distinto de cero; en otro caso utiliza teclado/gamepad y limita magnitud a 1. La física y las colisiones conservan su ruta existente en FixedUpdate, reconciliación del store y desplazamiento por ejes. No se aplica Transform.Translate desde el joystick.

Soltar el dedo propietario, desactivar el control, pausar, perder foco o cambiar de escena restablece vector y mango. Durante diálogo se deshabilita el controlador del receptor y se oculta la palanca. El HUD se oculta durante viajes y cuando no hay jugador en escena.

### 11.4 Botón contextual y extensión de zonas

`IMobileInteractionTarget` exige tres miembros:

```
bool CanInteractWith(PlayerMovementController player);
void InteractWith(PlayerMovementController player);
Vector3 InteractionPosition { get; }
```

`ModularInteraction` y `SceneTeleport` se registran en `MobileInteractionTargets` al habilitarse y se retiran al deshabilitarse. `Nearest(player)` filtra objetivos válidos y elige por distancia cuadrática al Transform de referencia; no mide el borde del collider. Los empates no tienen una prioridad explícita.

Fuera de un objetivo válido, E permanece oculto. Una misión automática sin diálogo no crea una acción E. Durante un diálogo, E llama a `AdvanceCurrent`; durante una recompensa se oculta porque ya existe Continuar táctil. Se impide actuar en el frame de cierre modal y se reinicia la palanca antes de interactuar.

El HUD busca periódicamente un controlador con `AcceptsLocalInput`; descarta uno que cambie a autoridad remota. Utiliza los EventSystem de las escenas. Las escenas nuevas necesitan EventSystem e InputSystemUIInputModule configurados para punteros táctiles independientes. Un canvas visible sin módulo UI no puede recibir arrastres ni clics.

## 12 Frontera de replicación e integración con main

`PlayerReplicationContext` define `OfflineLocal`, `LocalOwner` y `RemoteReplica`. `AcceptsLocal Input` es falso solo para la réplica remota. El controlador de movimiento comprueba esa autoridad antes de leer joystick, teclado o gamepad. Registra `IsMoving` en el store cuando cambia el estado de movimiento y lo limpia al deshabilitar un controlador local.

`PlayerReplicationBridge` ofrece `CaptureSnapshot`, `LocalSnapshotChanged` y `TryApplyRemote Snapshot`. Un snapshot remoto solo se aplica a una réplica remota y debe coincidir con su playerId configurado. El snapshot incluye datos replicables del jugador; no sustituye la configuración local de collider, Y-sort o duración de animación. El bridge no establece conexiones por sí mismo.

La integración local `e2eca4f` combinó autoridad de red y movimiento táctil, unificó las escenas antiguas en Deprecate preservando GUID y conservó la lista de salas y opciones WebGL. `1f1abe5` registra la integración del PR 7 en main; esta revisión ya parte de ese resultado, no de una rama en conflicto.

Los módulos `Scripts/Network` y `Scripts/Networking` conservan implementaciones recibidas de colaboradores. Hay código basado en UDP y en `System.Net.WebSockets.ClientWebSocket`; no se validó transporte completo en navegador. El puente JavaScript de controles móviles no adapta esos transportes. Antes de afirmar multijugador Web, definir y probar un transporte compatible, endpoint HTTPS/WSS y autorización del servidor.

## 13 Pantalla inicial y tipografía

El Header de InitialDemoScene utiliza el título **NIWI-MU** y el subtítulo **Selecciona tu rol para comenzar**. Sus rectángulos están separados y centrados, con anchors relativos al ancho, autoajuste y recorte Truncate. La organización de botones se conserva.

La fuente es `Asset/Fonts/PressStart2P/PressStart2PSDF.asset`; el TTF y su licencia se conservan en la misma familia de recursos. TMP usa esa fuente como predeterminada. Al redistribuir el juego, conservar el aviso de licencia correspondiente.

El aviso de ellipsis se debía a `TextOverflowModes.Ellipsis=1` en los dos textos del Header. Se reemplazó por `Truncate=3`. Esto elimina la dependencia del glifo de puntos suspensivos para ese recorte; no agrega glifos al atlas ni garantiza que cualquier texto Unicode futuro esté cubierto. Revisar caracteres, fallbacks y atlas al ampliar contenido.

# 14 Build Web y publicación

## 14.1 Build manual frente a constructor de demo

| Elemento | Contrato y comportamiento |
|---|---|
| Build Profiles / Build | Respeta el perfil y orden configurados. En la lista global revisada, InitialDemoScene es la primera escena habilitada. Adecuado para probar el flujo de roles completo. |
| `WebPreviewBuild.Run` | Antecede explícitamente Plaza Central a las escenas habilitadas existentes. Por diseño omite la selección de roles como primera pantalla. |
| `tools/build_web_preview.py` | Crea una copia de Assets, Packages y ProjectSettings bajo Builds/WebPreviewProject, conserva su Library y ejecuta el constructor de demo en batch. |

El constructor usa Gzip, Decompression Fallback, nombres con hash y `BuildOptions.None`. Restaura sus tres opciones WebGL en finally. El destino se toma de `CENIGRAF_WEB_OUTPUT` o de `Builds/WebGL`. No crea una cuenta de hosting ni publica por sí mismo.

El script Python lee la versión de ProjectVersion.txt; su ruta de editor por defecto corresponde a Unity Hub en macOS. En otro sistema se debe indicar `--editor`. Compila lo guardado en disco, no cambios de escena pendientes en el editor. No ejecutar dos compilaciones simultáneas sobre su misma copia temporal.

```
python3 tools/build_web_preview.py
python3 -m http.server 8080 --directory Builds/WebGL
```

Abrir `http://localhost:8080`. No usar file:// para validar WebAssembly. Para iniciar en el menú, utilizar el build manual o modificar conscientemente StartScene del constructor; cambiar el orden de Scene List no cambia ese StartScene.

## 14.2 Tamaño y despliegue externo

La build de prueba anterior tenía aproximadamente 22 MiB comprimidos: unos 12 MiB de WebAssembly y 10 MiB de datos. Es una medición histórica, anterior a nuevas compilaciones, no una garantía del tamaño de main actual. No equivale al tamaño de Assets, Library, archivos de símbolos, build nativo ni RAM utilizada al ejecutar.

La entrega Web necesita index.html y todas sus rutas relativas. El ZIP de publicación debe tener ese archivo en la raíz, junto a Build y TemplateData, sin una carpeta envolvente adicional. La guía `WEB_PREVIEW.md` describe compilación, servidor local, carga HTML en itch.io y diagnóstico. Toda modificación de código, assets o escenas requiere generar y subir otra build; el hosting no sincroniza automáticamente el editor.

# 15 Verificación y evidencia disponible

## 15.1 Comprobaciones realizadas durante las modificaciones

| Elemento | Contrato y comportamiento |
|---|---|
| Compilación y referencias | Unity 6000.3.18f1 en copia independiente; cuatro escenas habilitadas abiertas sin scripts faltantes ni referencias rotas detectadas. Resultado: `INTEGRATIONVALIDATIONPASSED`. |
| Frontera de replicación | Se verificó aplicación de campos replicables y preservación de configuración local del jugador. |
| Controles móviles | Prueba de integración en ejecución: vector táctil hacia movimiento, diagonal limitada, mango dentro del círculo, propiedad de puntero, reset, E contextual y avance/cierre de diálogo. |
| Jugador remoto | Prueba adicional: RemoteReplica no consume entrada local ni activa un área modular. Resultado conjunto del merge: dos pruebas aprobadas. |
| Layout simulado | Rectángulos de referencia 874×402, 402×874 y 1194×834; comprobación de dimensiones y separación de controles. |
| Puente JavaScript | Casos simulados de agentes iPhone, Android, iPad con agente de escritorio, Mac y laptop Windows táctil, además del ratio CSS y touch-action. |
| Build anterior | Carga de Plaza Central y respuesta al teclado en navegador local; publicación de esa build confirmada. No certifica que todo main actual ya esté desplegado. |

Las pruebas temporales de controles se ejecutaron en la copia de validación; no están incorporadas como una suite permanente en Assets ni como CI. Sus eventos de puntero son simulados: no equivalen a usar físicamente dos dedos en Safari/Chrome. Esta actualización documental no vuelve a ejecutar el juego ni publica un build nuevo.

## 15.2 Verificación reproducible del repositorio

El validador versionado se ejecuta desde Unity en batch sobre una copia del proyecto:

```
"$UNITY_EDITOR" -batchmode -nographics -quit \
  -projectPath "$PROJECT_ROOT" \
  -executeMethod SocialProyectoCenigraf.Editor.IntegrationValidation.Run \
  -logFile /tmp/niwi-integration.log
```

Definir ambas variables con rutas reales. No abrir la misma carpeta simultáneamente en otro editor. El validador recorre escenas habilitadas y comprueba referencias; las aserciones específicas de SceneDemo

solo se ejecutan si esa escena está incluida. No prueba audio perceptual, servidor, dispositivos reales ni todo el flujo de teleport.

### 15.3 Matriz manual antes de una publicación

1. Inicio manual: Visitante/Aprendiz llegan a Plaza; Docente/Administrativo abren submenú y luego Ir al juego. Confirmar diferencias de permiso de publicaciones.
2. Diálogo: apertura, escritura, completar texto, terminar y cancelar sin recompensa. E no debe reabrir en el mismo frame.
3. Carnet: recibir, cancelar ventana, reabrir pendiente, confirmar rol Aprendiz y cambiar de escena. Verificar que no se duplique recepción.
4. Teleport: ida/vuelta, rol permitido/rechazado, ID inexistente/duplicado y escena fuera de build. El spawn inicial no debe verse antes del resuelto.
5. Móvil: horizontal/vertical, notch, cambio de orientación, dos dedos, soltar fuera de base, pérdida de foco y regreso. Fuera de áreas no debe aparecer E.
6. Audio: volumen moderado, primer gesto en navegador, hover/clic de menús generados y recompensa sin repetición de melodía al reabrir.
7. Rendimiento: tiempo de primera carga, memoria, FPS y recuperación tras varias transiciones. Registrar dispositivo, navegador, build y resultado.

# 16 Pendientes priorizados y mantenimiento

| Elemento | Contrato y comportamiento |
|---|---|
| Alta: pruebas físicas | Verificar iOS/Android/tablet con la build realmente publicada. La responsividad del HUD no certifica todos los paneles ni el rendimiento del juego completo. |
| Alta: pruebas versionadas | Incorporar fixtures y aserciones de interacción/autoridad a Test Framework y CI; hoy parte de la evidencia depende de pruebas temporales de sesión. |
| Alta: autorización y red | Verificar transporte Web y reglas en servidor. Roles locales y exclusiones UI pueden alterarse en el cliente. |
| Alta: llegada multijugador | Sustituir selección del primer jugador del gate por resolución explícita del jugador local/ID antes de utilizar teleports con múltiples jugadores activos. |
| Media: misiones | Añadir estados de progreso/completada, eventos de objetivos, recompensas idempotentes y visualización. |
| Media: persistencia | Diseñar guardado/versionado de misiones, recompensas, rol e inventario; actualmente solo hay memoria de sesión. |
| Media: edición de destino | Exponer el destino Ir al juego del submenú como configuración si se desea evitar cambios de código. |
| Media: accesibilidad | Ajuste de volumen en opciones, escalado/reposición de controles, contraste, preferencias de texto y revisión de todos los glifos necesarios. |

Al cambiar rutas, conservar archivos .meta/GUID, revisar referencias y escenas del build. Al agregar otra zona interactuable, implementar el contrato móvil y registrar/desregistrar el componente. Al extender estados, mantener las fábricas, reducer, métodos With y comparación de igualdad coherentes. Actualizar esta fecha/base y la matriz de cobertura cuando cambie una capacidad o su validación.